SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO3-CASE STUDY

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

(a) Understand the Case

1. Introduction

KartIQ is a national e-commerce platform that processes approximately 10,000,000 orders per day and ranks 100,000 sellers. Its existing system faces several performance problems:

- Nightly order reports are delayed because sorting takes too long.
- The seller leaderboard freezes during major sales.
- Customer-support staff experience slow order lookups.
- Product recommendation scores are refreshed too infrequently.

The platform can improve its performance using divide-and-conquer, a technique that divides a large problem into smaller subproblems, solves them, and combines their results.

The four modules require different algorithms because their data and performance requirements are different.

2. Analysis table

Module	Computational requirement	Key issues	Suitable algorithm
1. Nightly Order Consolidation	Sort 10 million orders by timestamp	Stability, large input size, memory usage	Merge Sort
2. Live Seller Leaderboard	Re-sort 100,000 seller scores frequently	Nearly sorted data, poor pivot selection, worst-case time	Quick Sort with randomized or median-of-three pivot
3. Order Lookup	Find an order by timestamp	Data must be sorted; first matching timestamp is required	Modified Binary Search

4. Recommendation Scores	Multiply two 1024×1024matrices	Large number of scalar multiplications and computation time	Strassen's Matrix Multiplication

3. Why algorithm selection matters

- Module 1 – Merge Sort: A stable sorting algorithm is necessary because orders with identical timestamps must retain their original arrival order. A poor sorting method may produce an incorrect audit sequence or miss the reporting deadline.
- Module 2 – Quick Sort: Choosing the first element as the pivot can create highly unbalanced partitions. This may increase execution time from $O(n \log n)$to $O(n^2)$, causing the leaderboard to freeze.
- Module 3 – Binary Search: Binary Search requires sorted data. Applying it to an unsorted list can return an incorrect result. Ordinary Binary Search may also return any matching order instead of the first one.
- Module 4 – Strassen's Method: Conventional matrix multiplication requires $O(n^3)$scalar multiplications. Strassen's method reduces the asymptotic multiplication count, but its additional additions, memory usage, and implementation overhead must also be considered.

(b) Module 1 – Merge Sort for Nightly Order Consolidation

1. Aim

To sort orders in ascending timestamp order while preserving the original arrival order of orders with equal timestamps.

Given input

Each order contains a timestamp and an order ID.

Input orders:

(105, O1), (102, O2), (105, O3),

(101, O4), (102, O5), (103, O6)

The expected output is sorted by timestamp. For equal timestamps, the earlier-arriving order must appear first.

2. Working principle of Merge Sort

Merge Sort follows three steps.

1. Divide: Divide the list into two smaller halves repeatedly until each sublist contains one order.
2. Conquer: A sublist containing one element is already sorted.
3. Combine: Merge the sorted sublists by repeatedly selecting the order with the smaller timestamp.

When two timestamps are equal, the merge operation must select the element from the left sublist first. This preserves stability.

3. Algorithm

MERGE_SORT(A):

If the size of A is less than or equal to 1:

Return A

Divide A into left and right halves

left = MERGE_SORT(left)

right = MERGE_SORT(right)

Return MERGE(left, right)

The merge procedure compares timestamps and combines the two sorted halves. If the timestamps are equal, it selects the left element first.

4. Step-by-step trace

Step 1: Original list

```
[(105,O1), (102,O2), (105,O3),
 (101,O4), (102,O5), (103,O6)]
```

Step 2: Divide into halves

```
Left:
[(105,O1), (102,O2), (105,O3)]

Right:
[(101,O4), (102,O5), (103,O6)]
```

Divide the left half:

```
[(105,O1)]    [(102,O2), (105,O3)]
```

Divide the right half:

```
[(101,O4)]    [(102,O5), (103,O6)]
```

Divide the remaining pairs:

```
[(102,O2)] [(105,O3)]
[(102,O5)] [(103,O6)]
```

All sublists now contain one element.

Step 3: Merge the smaller sublists

Merge the second and third orders from the left side:

```
[(102,O2)] + [(105,O3)]

Result: [(102,O2), (105,O3)]
```

Merge the corresponding right-side pair:

```
[(102,O5)] + [(103,O6)]

Result: [(102,O5), (103,O6)]
```

Step 4: Merge the left half

Merge:

```
[(105,O1)]
[(102,O2), (105,O3)]
```

Comparisons:

- Compare 105 and 102. Select (102, O2).
- Compare 105 and 105. Both timestamps are equal, so select the left element (105, O1) first.
- Append the remaining (105, O3).

Result:

```
[(102,O2), (105,O1), (105,O3)]
```

Notice that O1 appears before O3, preserving their original arrival order.

Step 5: Merge the right half

Merge:

```
[(101,O4)]
[(102,O5), (103,O6)]
```

Comparisons:

- Compare 101 and 102. Select (101, O4).
- Append (102, O5) and (103, O6).

Result:

```
[(101,O4), (102,O5), (103,O6)]
```

Step 6: Final merge

Merge the two sorted halves:

```
Left:
[(102,O2), (105,O1), (105,O3)]

Right:
[(101,O4), (102,O5), (103,O6)]
```

Comparison	Selected order
102 vs 101	(101, O4)
102 vs 102	(102, O2), from the left
105 vs 102	(102, O5)
105 vs 103	(103, O6)

Left elements remaining	(105, O1), (105, O3)

Final sorted list:

```
[(101,O4), (102,O2), (102,O5),
 (103,O6), (105,O1), (105,O3)]
```

Conclusion: The orders are sorted in ascending timestamp order, and the equal-timestamp orders retain their original relative order.

5. Python program for stable Merge Sort

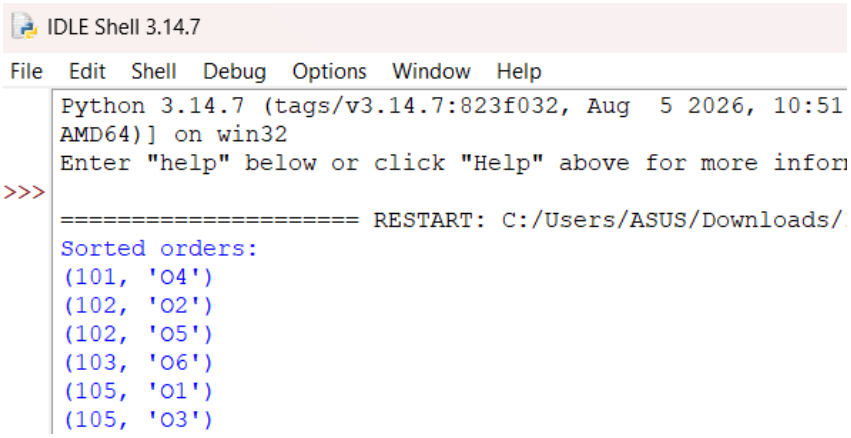
```
def merge_sort(orders):
    if len(orders) <= 1:
        return orders
    middle = len(orders) // 2
    left = merge_sort(orders[:middle])
    right = merge_sort(orders[middle:])
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i][0] <= right[j][0]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

orders = [
    (105, "O1"), (102, "O2"), (105, "O3"),
    (101, "O4"), (102, "O5"), (103, "O6")
]
sorted_orders = merge_sort(orders)
print("Sorted orders:")
for order in sorted_orders:
    print(order)
```

Output



Explanation of the code

- merge_sort() divides the list into two halves.
- Recursive calls sort both halves.
- The while loop compares timestamps and merges the orders.
- The <= condition ensures that the left-side order is selected first when timestamps are equal.

- extend() adds the remaining elements after one half becomes empty.

6. Recurrence relation and solution

Let $T(n)$represent the time required to sort $n$orders.

Merge Sort divides the input into two halves and spends linear time merging them.

$$T(n) = 2T(n/2) + cn$$

where cnrepresents the time taken to merge the elements.

Expanding the recurrence:

$$T(n) = 2[2T(n/4) + cn/2] + cn$$

$$T(n) = 4T(n/4) + 2cn$$

After $k$levels:

$$T(n) = 2^k T(n/2^k) + kcn$$

At the final level, $n/2^k = 1$, so $k = \log_2 n$.

Therefore,

$$T(n) = nT(1) + cn\log_2 n$$

Hence, the time complexity is:

$$\boxed{T(n) = O(n \log n)}$$

The best, average and worst cases are all $O(n \log n)$.

7. Estimated comparisons for 10,000,000 orders

Given:

$$n = 10{,}000{,}000$$

$$\log_2 n \approx 23.2535$$

Using $n\log_2 n$as an approximate comparison estimate:

$$10{,}000{,}000 \times 23.2535$$

$$\boxed{\approx 232.5 \text{ million comparisons}}$$

This is an asymptotic estimate, not an exact count. For example, the standard upper bound for binary Merge Sort is approximately 223.2 million comparisons for this input size.

The actual number depends on the arrangement of the input and how the merge operation handles the remaining elements.

8. Memory requirement

Merge Sort requires additional memory to merge the sublists.

- Time complexity: $O(n \log n)$
- Auxiliary array memory: $O(n)$
- Recursion stack: $O(\log n)$

For 10 million orders, memory usage can be substantial. If an order record requires $b$bytes, storing all records alone requires approximately $10^7 b$bytes, excluding temporary arrays and other program overhead.

Handling data that does not fit into RAM

KartIQ can use External Merge Sort:

1. Read a manageable block of orders into RAM.
2. Sort the block using Merge Sort.
3. Write the sorted block to disk as a sorted run.
4. Repeat until all orders have been processed.
5. Merge the sorted runs using buffered disk reads and a priority queue.

This approach processes large datasets without requiring the entire dataset to fit in memory.

Justification: Stable Merge Sort provides predictable $O(n \log n)$performance and can be extended to external storage, making it appropriate for nightly order consolidation.

(c) Module 2 – Quick Sort for the Live Seller Leaderboard

1. Aim

To sort seller scores efficiently during frequent leaderboard refreshes and avoid the poor performance caused by selecting the first element as the pivot.

Given input

42, 17, 63, 8, 55, 29

The current implementation selects the first element, 42, as the pivot.

2. Working principle of Quick Sort

Quick Sort follows three main steps:

1. Choose a pivot: Select one element from the list.
2. Partition: Place elements smaller than the pivot on the left and larger elements on the right.
3. Conquer: Recursively sort the two partitions.

Unlike Merge Sort, Quick Sort usually sorts the array in place and requires relatively little auxiliary memory.

3. Trace of one partition step

Initial array:

[42, 17, 63, 8, 55, 29]

Choose pivot:

Pivot = 42

Using the first element as the pivot, partition the remaining elements according to whether they are smaller or larger than 42.

Element	Comparison	Action
17	17 < 42	Move to the left partition
63	63 > 42	Keep in the right partition
8	8 < 42	Move to the left partition
55	55 > 42	Keep in the right partition
29	29 < 42	Move to the left partition

After partitioning, one valid partitioned array is:

[17, 8, 29, 42, 55, 63]

The pivot 42 is in its final sorted position. The left partition is [17, 8, 29], and the right partition is [55, 63]. The left and right partitions must still be sorted recursively.

4. Python program for Quick Sort

The following code uses the first element as the pivot to demonstrate the current implementation.

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]

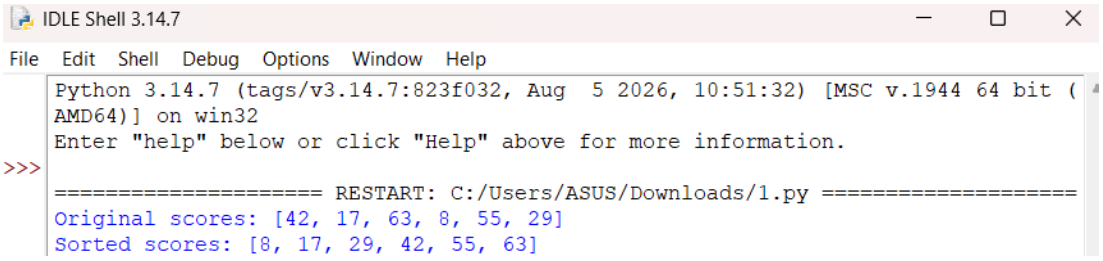
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

scores = [42, 17, 63, 8, 55, 29]

print("Original scores:", scores)
print("Sorted scores:", quick_sort(scores))
```

Output



5. Why the first-element pivot makes the leaderboard slow

The first-element pivot works reasonably well when it divides the array into balanced partitions. However, when the array is already sorted or nearly sorted, the pivot can repeatedly produce highly unbalanced partitions.

For example, consider:

[8, 17, 29, 42, 55, 63]

If the first element is always selected as the pivot, the partitions may have sizes:

0 elements and 5 elements

0 elements and 4 elements

0 elements and 3 elements

...

The recursion becomes deep, and many comparisons are repeated.

For $n$elements, the worst-case number of comparisons is approximately:

$$(n-1)+(n-2)+\cdots+1$$

$$=\frac{n(n-1)}{2}$$

This is quadratic time, $O(n^2)$. At 100,000 sellers, this can lead to billions of comparisons and make the live leaderboard unresponsive.

6. Best-case recurrence and solution

In the best case, each pivot divides the array into two approximately equal halves.

$$T(n)=2T(n/2)+cn$$

Expanding:

$$T(n)=4T(n/4)+2cn$$

After $k$levels:

$$T(n)=2^kT(n/2^k)+kcn$$

Since $k=\log_2 n$,

$$\boxed{T(n)=O(n\log n)}$$

7. Worst-case recurrence and solution

In the worst case, one partition has $n-1$elements and the other has zero elements.

$$T(n) = T(n-1) + cn$$

Expanding:

$$T(n) = T(n-2) + c(n-1) + cn$$

Continuing:

$$T(n) = T(1) + c(2 + 3 + \cdots + n)$$

Using the sum formula:

$$T(n) = T(1) + c\left(\frac{n(n+1)}{2} - 1\right)$$

Therefore,

$$\boxed{T(n) = O(n^2)}$$

The average-case complexity of randomized Quick Sort is $O(n \log n)$, while its worst-case complexity remains $O(n^2)$.

8. Estimated comparisons for 100,000 sellers

Given:

$$n = 100{,}000$$

For a balanced partitioning pattern:

$$n\log_2 n = 100{,}000 \times 16.6094$$

$$\boxed{\approx 1.66 \text{ million comparisons}}$$

For the worst case:

$$\frac{n(n-1)}{2} = \frac{100{,}000 \times 99{,}999}{2}$$

$$\boxed{4{,}999{,}950{,}000 \text{ comparisons}}$$

The estimates show why pivot selection matters. These are theoretical comparison counts, not measured execution times.

9. Improved pivot strategy

KartIQ should use median-of-three pivot selection, which selects the median of the first, middle and last elements.

For the sample:

First element = 42

Middle element = 63

Last element = 29

After ordering these three values:

29, 42, 63

The median is 42, so it becomes the pivot for this particular example. Although the sample's pivot does not change, the strategy can reduce poor pivot choices across other input arrangements.

A randomized pivot is another useful option. For stronger protection against bad partitions, KartIQ can use Introsort, which switches to Heap Sort if Quick Sort's recursion becomes too deep. Because the scores change only slightly between refreshes, an insertion-sort optimization for small partitions can also help.

Justification: A better pivot strategy reduces the likelihood of highly unbalanced partitions. Introsort additionally provides $O(n \log n)$worst-case time. This makes the leaderboard more reliable during sales events.

(d) Module 3 – Binary Search for Order Lookup

1. Aim

To search for an order by timestamp in the sorted order list and return the first occurrence when multiple orders have the same timestamp.

2. Why sorted data is essential

Module 1 produces the following sorted list:

```
[(101,04), (102,02), (102,05),
 (103,06), (105,01), (105,03)]
```

Binary Search works by comparing the target timestamp with the middle element.

- If the target is smaller, search the left half.
- If the target is larger, search the right half.
- If the target is equal, record the match and continue searching left to find the first occurrence.

An unsorted list cannot reliably be searched using Binary Search because the comparison does not indicate which half contains the target.

3. Working principle

Suppose the target timestamp is 102.

Initial list:

```
Index:      0      1      2      3      4      5
Timestamp: 101    102    102    103    105    105
```

Step-by-step trace

Step 1: Set low = 0, high = 5.

$$mid = \lfloor (0 + 5)/2 \rfloor = 2$$

The timestamp at index 2 is 102. It matches the target. Record index 2 and continue searching to the left.

Step 2: Set high = mid - 1 = 1.

$$mid = \lfloor (0 + 1)/2 \rfloor = 0$$

The timestamp at index 0 is 101. Since 101 is smaller than 102, search to the right by setting low = 1.

Step 3: Now low = 1, high = 1.

The middle index is 1, and its timestamp is 102. Record index 1 and continue searching left.

Step 4: Set high = 0. Now low > high, so the search stops.

The first occurrence is at index 1, corresponding to order O2.

4. Python program for finding the first occurrence

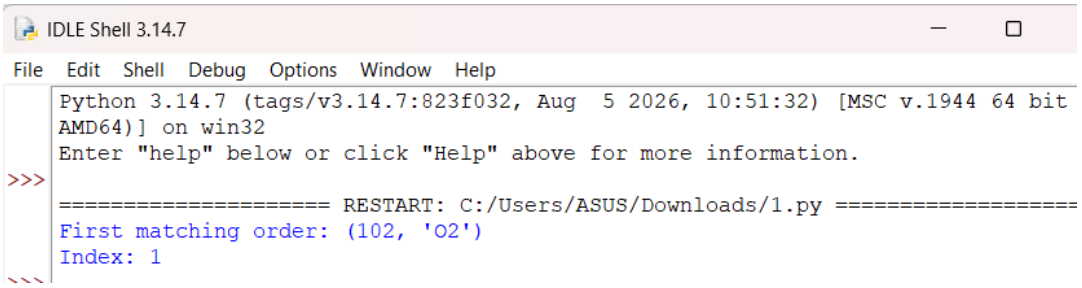
```python
def first_occurrence(orders, target):
    low = 0
    high = len(orders) - 1
    result = -1

    while low <= high:
        middle = (low + high) // 2
        if orders[middle][0] == target:
            result = middle
            high = middle - 1
        elif orders[middle][0] < target:
            low = middle + 1
        else:
            high = middle - 1

    return result

orders = [
    (101, "O4"), (102, "O2"), (102, "O5"),
    (103, "O6"), (105, "O1"), (105, "O3")
]
target = 102
index = first_occurrence(orders, target)
if index != -1:
    print("First matching order:", orders[index])
    print("Index:", index)
else:
    print("Order not found")
```

Output



5. Recurrence relation and solution

At each step, Binary Search reduces the search space to approximately half its previous size.

$$T(n) = T(n/2) + c$$

Expanding:

$$T(n) = T(n/4) + 2c$$

After $k$levels:

$$T(n) = T(n/2^k) + kc$$

When the remaining search space contains one element:

$$n/2^k = 1$$

Therefore,

$$k = \log_2 n$$

Substituting:

$$T(n) = T(1) + c\log_2 n$$

Hence,

$$\boxed{T(n) = O(\log n)}$$

The modified algorithm that finds the first occurrence still has $O(\log n)$time complexity because it continues to reduce the search interval by half after finding a match.

6. Maximum comparisons for 10,000,000 orders

Given:

$$n = 10{,}000{,}000$$

$$\log_2(10,000,000) \approx 23.2535$$

The number of comparisons is logarithmic. The maximum number of iterations for this input size is at most 24, with the exact number of comparisons depending on the search implementation and target.

$$\boxed{\text{Maximum iterations} \leq 24}$$

Thus, instead of checking up to 10 million orders sequentially, Binary Search can locate a timestamp using approximately 24 iterations.

7. Complexity analysis

Case	Time complexity
Best case	$O(1)$
Average case	$O(\log n)$
Worst case	$O(\log n)$
Iterative auxiliary memory	$O(1)$

Justification: Binary Search is suitable because Module 1 already supplies a sorted list. The modified version meets the additional requirement of returning the earliest matching timestamp, which is important for customer-support investigations and audit records.

(e) Module 4 – Strassen's Method for Recommendation Scores

1. Aim

To calculate the product of two matrices efficiently using Strassen's divide-and-conquer method.

KartIQ computes recommendation scores using:

$$C = A \times B$$

The given matrices are:

$$A = \begin{bmatrix} 1 & 7 \\ 3 & 5 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 3 \\ 7 & 5 \end{bmatrix}$$

In production, both matrices have dimensions \(1024\times1024\).

2. Conventional matrix multiplication

For two 2×2 matrices,

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

the result is:

$$C = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

This requires eight scalar multiplications for a 2×2 matrix.

For the given matrices:

$$C_{11} = (1 \times 1) + (7 \times 7) = 50$$
$$C_{12} = (1 \times 3) + (7 \times 5) = 38$$
$$C_{21} = (3 \times 1) + (5 \times 7) = 38$$
$$C_{22} = (3 \times 3) + (5 \times 5) = 34$$

Therefore,

$$\boxed{C = \begin{bmatrix} 50 & 38 \\ 38 & 34 \end{bmatrix}}$$

3. Working principle of Strassen's method

Strassen's method divides each input matrix into four submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$
$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Instead of performing eight recursive matrix multiplications, it performs seven. Additional matrix additions and subtractions are used to reconstruct the final answer.

For 2×2 example, each submatrix is a single number.

4. The seven Strassen products

The seven products are:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$
$$M_2 = (A_{21} + A_{22})B_{11}$$
$$M_3 = A_{11}(B_{12} - B_{22})$$
$$M_4 = A_{22}(B_{21} - B_{11})$$
$$M_5 = (A_{11} + A_{12})B_{22}$$
$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

5. Reconstruction formulas

The final result is reconstructed using:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$C_{12} = M_3 + M_5$$
$$C_{21} = M_2 + M_4$$
$$C_{22} = M_1 - M_2 + M_3 + M_6$$

These formulas combine the seven products to obtain the same result as conventional matrix multiplication.

6. Verification using the sample matrices

The individual matrix entries are:

$$A_{11} = 1,\ A_{12} = 7,\ A_{21} = 3,\ A_{22} = 5$$

$$B_{11} = 1,\ B_{12} = 3,\ B_{21} = 7,\ B_{22} = 5$$

Calculate each product.

Product	Calculation	Result
M_1	$(1 + 5)(1 + 5) = 6 \times 6$	36
M_2	$(3 + 5) \times 1$	8
M_3	$1 \times (3 - 5)$	-2
M_4	$5 \times (7 - 1)$	30
M_5	$(1 + 7) \times 5$	40
M_6	$(3 - 1)(1 + 3)$	8
M_7	$(7 - 5)(7 + 5)$	24

Reconstruct the result

First element:

$$C_{11} = 36 + 30 - 40 + 24 = 50$$

Second element:

$$C_{12} = -2 + 40 = 38$$

Third element:

$$C_{21} = 8 + 30 = 38$$

Fourth element:

$$C_{22} = 36 - 8 - 2 + 8 = 34$$

Thus,

$$\boxed{C = \begin{bmatrix} 50 & 38 \\ 38 & 34 \end{bmatrix}}$$

7. Python program for Strassen's method

The following program implements Strassen's algorithm for square matrices whose dimension is a power of two. It uses conventional multiplication as the base case for 1×1 matrices.

```
def add(A, B):
    n = len(A)
    return [
        [A[i][j] + B[i][j] for j in range(n)]
        for i in range(n)
    ]
def subtract(A, B):
    n = len(A)
    return [
        [A[i][j] - B[i][j] for j in range(n)]
        for i in range(n)
    ]
def strassen(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0] * B[0][0]]]
    middle = n // 2
```

```
    A11 = [row[:middle] for row in A[:middle]]
    A12 = [row[middle:] for row in A[:middle]]
    A21 = [row[:middle] for row in A[middle:]]
    A22 = [row[middle:] for row in A[middle:]]

    B11 = [row[:middle] for row in B[:middle]]
    B12 = [row[middle:] for row in B[:middle]]
    B21 = [row[:middle] for row in B[middle:]]
    B22 = [row[middle:] for row in B[middle:]]

    M1 = strassen(add(A11, A22), add(B11, B22))
    M2 = strassen(add(A21, A22), B11)
    M3 = strassen(A11, subtract(B12, B22))
    M4 = strassen(A22, subtract(B21, B11))
    M5 = strassen(add(A11, A12), B22)
    M6 = strassen(subtract(A21, A11), add(B11, B12))
    M7 = strassen(subtract(A12, A22), add(B21, B22))

    C11 = add(subtract(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(subtract(add(M1, M3), M2), M6)
    top = [
        C11[i] + C12[i] for i in range(middle)
    ]
    bottom = [
        C21[i] + C22[i] for i in range(middle)
    ]

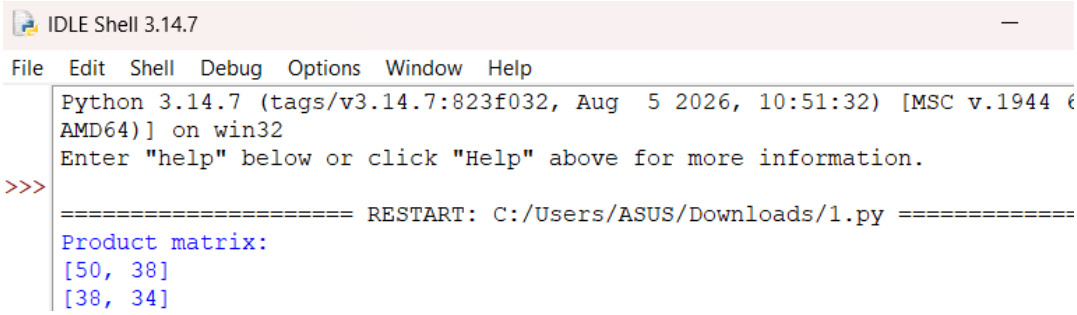
    return top + bottom


A = [[1, 7], [3, 5]]
B = [[1, 3], [7, 5]]

C = strassen(A, B)

print("Product matrix:")
for row in C:
    print(row)
```

Output



8. Recurrence relation and solution

Let $T(n)$ represent the time required to multiply two $n \times n$ matrices using Strassen's method.

The algorithm performs seven recursive multiplications on matrices of size $n/2$. The matrix additions and subtractions take $O(n^2)$ time.

Therefore,X

$$T(n) = 7T(n/2) + cn^2$$

Expanding:

$$T(n) = 7[7T(n/4) + c(n/2)^2] + cn^2$$

$$T(n) = 49T(n/4) + \frac{7cn^2}{4} + cn^2$$

At each level, the recursive multiplication contributes a factor of seven, while the matrix size is halved.

Using the Master Theorem:

- $a = 7$
- $b = 2$
- $f(n) = cn^2$

Calculate:

$$n^{\log_b a} = n^{\log_{27}}$$

$$\log_{27} \approx 2.8074$$

Since n^2 grows more slowly than $n^{2.8074}$, the recurrence gives:

$$\boxed{T(n) = O\left(n^{\log_{27}}\right)}$$

$$\boxed{T(n) \approx O(n^{2.8074})}$$

This is asymptotically faster than conventional matrix multiplication, which requires $O(n^3)$ time.

9. Comparison of scalar multiplications for $n = 1024$

Given:

$$n = 1024 = 2^{10}$$

Conventional matrix multiplication

The number of scalar multiplications is:

$$n^3 = 1024^3$$

$$\boxed{1{,}073{,}741{,}824}$$

Strassen's method

Each recursive level uses seven multiplications. For a matrix dimension of 2^{10} , the recursion has ten levels before reaching scalar multiplications.

$$7^{10} = 282{,}475{,}249$$

$$\boxed{282{,}475{,}249 \text{ scalar multiplications}}$$

Comparison table

Method	Scalar multiplications	Asymptotic time
Conventional multiplication	1,073,741,824	$O(n^3)$
Strassen's method	282,475,249	$O(n^{2.8074})$

The theoretical reduction is:

$$1{,}073{,}741{,}824 - 282{,}475{,}249$$

$$= 791{,}266{,}575$$

Percentage reduction:

$$\frac{791{,}266{,}575}{1{,}073{,}741{,}824} \times 100$$

$$\boxed{\approx 73.69\%}$$

Strassen's method uses approximately 73.69% fewer scalar multiplications for this matrix size under the standard recursive operation-count model.

10. Why Strassen's method may not be faster in practice

Despite its lower asymptotic complexity, Strassen's method may not always be faster for real applications.

1. Additional additions and subtractions: Each recursive level requires extra matrix operations.
2. Memory usage: Temporary matrices are created during recursive calculations.
3. Recursion overhead: Repeated function calls introduce overhead.
4. Numerical accuracy: For floating-point matrices, repeated additions and subtractions can increase rounding errors.
5. Hardware efficiency: Conventional matrix multiplication can use highly optimized libraries and processor instructions.
6. Small matrix sizes: For small matrices, conventional multiplication can be faster because it avoids recursive overhead.

11. Practical hybrid solution

KartIQ should use a hybrid matrix multiplication strategy.

- Use Strassen's method for sufficiently large matrices.
- Switch to conventional matrix multiplication when the matrix dimension falls below a chosen threshold.
- Use an optimized numerical library for the conventional multiplication step.
- Select the threshold through benchmarking on the production hardware.
- Allocate temporary buffers efficiently to reduce memory allocation overhead.

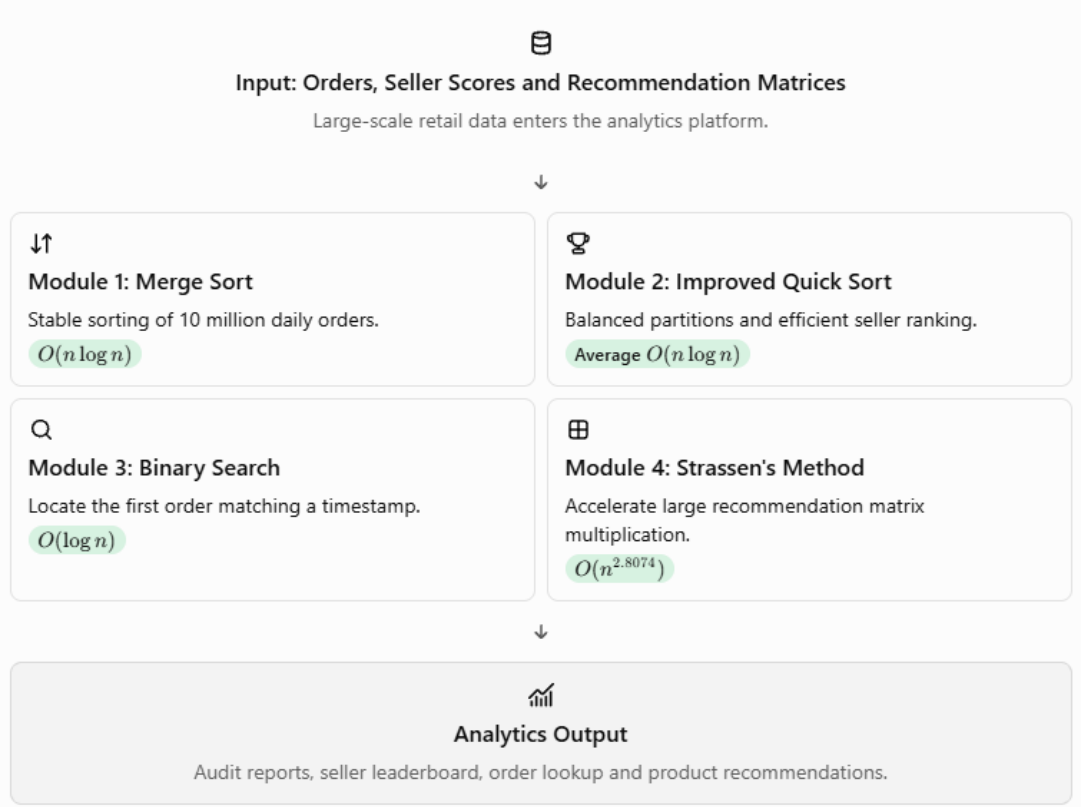
For example, the implementation can recursively use Strassen's method until the submatrices become sufficiently small, then finish with conventional multiplication.

Justification: This approach benefits from Strassen's reduced multiplication count for large matrices while avoiding excessive recursion and temporary allocations for small matrices.

(f) Design and Justify the Final Solution

1. Final algorithmic design for KartIQ

The following design combines the four algorithms into one practical architecture.



2. Final comparison table

Module	**Selected algorithm**	**Time complexity**	**Extra memory**	**Main justification**
Nightly Order Consolidation	Stable Merge Sort	$O(n \log n)$	$O(n)$	Preserves the arrival order of equal timestamps
Live Seller Leaderboard	Quick Sort with median-of-three or randomized pivot; use Introsort for worst-case protection	Expected $O(\text{n} \log n)$; Introsort guarantees $O(n \log n)$	$O(\log n)$ typical stack usage; bounded stack with Introsort	Reduces poor partitions and protects refresh performance
Order Lookup	Modified Binary Search	$O(\log n)$	$O(1)$	Finds the first matching timestamp in sorted data
Recommendation Scores	Hybrid Strassen's Method	$O(n^{2.8074})$ asymptotically	$O(n^2)$ working memory in a straightforward	Reduces recursive multiplication

			implementation	count for large matrices

Memory figures are auxiliary-memory estimates for conventional implementations. Actual usage depends on implementation details, recursion strategy and buffer management.

3. Main trade-offs and conditions for changing each decision

Module	Main trade-off	When the choice should change
Merge Sort	Requires additional memory	Use External Merge Sort when the dataset cannot fit into RAM
Quick Sort	Performance depends on partitioning unless worst-case safeguards are added	Use Introsort when predictable worst-case time is essential
Binary Search	Requires data to be sorted	Use a database index or another lookup structure if data changes frequently or is not maintained in sorted order
Strassen's Method	Fewer multiplications but more additions and temporary storage	Use conventional optimized multiplication when matrices are small or benchmarking shows it is faster

4. Final justification

The four modules have different computational requirements, so a single algorithm cannot efficiently solve all of them.

Merge Sort is suitable for order consolidation because it provides stable and predictable sorting. Improved Quick Sort makes seller-score sorting efficient on average, while Introsort offers a worst-case safeguard. Binary Search provides fast order lookups, provided the list remains sorted, and its modification ensures that the earliest matching order is returned. Strassen's method reduces the number of scalar multiplications required for large recommendation matrices, while a hybrid implementation accounts for practical overhead.

Conclusion

The KartIQ case study demonstrates how divide-and-conquer algorithms can improve the performance of a large retail data analytics platform.

- Merge Sort provides stable sorting in $O(n \log n)$ time.
- Improved Quick Sort efficiently ranks seller scores, with Introsort providing a worst-case $O(n \log n)$ guarantee.
- Modified Binary Search finds the first matching timestamp in $O(\log n)$ time.
- Strassen's Method reduces the asymptotic complexity of matrix multiplication to $O(n^{2.8074})$.